USING NDE APIS (DRAFT)

Last Updated: 07/05/2018
Submit Feedback: Dev Portal Slack channel#devportal

This guide provides general information about using NDe (Nike Digital engineering) APIs, including common standards, conventions, tips, and other helpful info that applies across multiple domains. Read this guide before diving into one of the per-API or per-domain Developer's Guides.

> **TIP:** Also check out the API Basics course offered by NDe Architecture team.

In This Guide:

Industry Standards

Learn how NDe APIs were designed with industry standards in mind.

REST Architecture

NDe uses the REST (**RE**presentational **S**tate **T**ransfer) architectural style, which allows you to communicates with our APIs over the Web using standard commands and protocols such as HTTP requests and responses.

REST is thoroughly explained on the web already (e.g. here), but here are a few reasons why we use it:

Stateless for Improved Performance

REST APIs are stateless, meaning that application state is maintained on the client and not within the API itself. Each interaction with the API is done in complete isolation: your requests must always include all of the necessary information to be fulfilled by the server. In turn, the server responses must always provide all necessary information you need to create state in your app.

Why is this desirable? Being stateless allows an API to be scaled across multiple servers and

therefore serve millions of concurrent users. It also allows for easy caching, which can improve performance.

Easy Adoption = Wide Adoption

REST is widely used in the industry because the syntax and protocols used (HTTP, JSON, the URI addressing protocol) are how you already use the web. **This means less work for you to to get your app up and running**.

JSON-formatted HTTP Requests and Responses

The standard format for exchanging data with NDe APIs is JSON (**J**ava**S**cript **O**bject **N**otation). As such, all HTTP request and response payloads must be in JSON format.

Wikipedia summarizes the benefits well: “JSON is a language-independent data format. It was derived from JavaScript, but as of 2017 many programming languages include code to generate and parse JSON-format data.”

Example of a JSON-formatted request body that was sent to a NDe API:

```
{
     "id":
"9af84d4b-6a1f-4899-af57-b4379f966913",
     "country": "US",
     "currency": "USD",
     "brand": "NIKE",
     "channel": "NIKECOM",
     "items": [
       {
         "id": "9892b8cb-e4ac-42af-
a8bb-3454d8509d32",
         "skuId":
"d3d15345-63e6-4cf9-9135-c2beacbbe352",
         "quantity": 2,
```

```
        "valueAddedServices": [
          {
            "id":
"88d0475d-30e5-4c2f-9a87-7a31938f8ace",
            "instruction": {
              "id": "2027261230",
              "type": "customization/nike_id"
            }
          },
          {
            "id":
"ac9fd9d3-3815-44ce-8b1f-18cce7abd488",
            "instruction": {
              "id": "1027551960",
              "type": "customization/nike_id"
            }
          }
        ]
      },
      {
        "id": "e945e0fd-
cdcf-4005-9fe4-9303b9d6235c",
        "skuId":
"f030bb22-4859-404e-82c9-aa9a23582e4f",
        "quantity": 1
      }
    ]
}
```

JSON Schema Helps Define API Contracts

The structures of the request and response bodies for NDe APIs are defined in each contract (an API.md file, commonly) using JSON Schema.

Per Wikipedia: “JSON Schema specifies a JSON-based format to define the structure of JSON data for validation, documentation, and interaction control. It provides a contract for the JSON data required by a given application, and how that data can be modified.”

Use the schema to understand the mandatory fields, expected data types, min/max values, and more in order to create requests and responses in accordance with the API contract.

For example the schema for the request body above is:

```
{   "$schema": "http://json-schema.org/
draft-04/schema#",
    "type": "object",
    "properties": {
      "id": {
        "type": "string",
        "description": "Shopping cart
unique identifier."
      },
      "resourceType": {
        "type": "string",
```

```
            "description": "The type of
resource the document is modeling.",
            "readonly": "create-update",
            "enum": [
              "cart"
            ]
          },
          "links": {
            "type": "object",
            "description": "Collection of links
to related resources.",
            "readonly": "create-update",
            "properties": {
              "self": {
                "type": "object",
                "description": "Link to this
resource, itself.",
                "readonly": "create-update",
                "properties": {
                  "ref": {
                    "type": "string",
                    "readonly": "create-update"
                  }
                },
                "required": [
                  "ref"
                ]
              }
            },
            "additionalProperties": true,
```

```
          "required": [
            "self"
          ]
        },
        "country": {
          "type": "string",
          "description": "2-alpha character
ISO 3166 country code.",
          "pattern": "^[A-Z]{2}$"
        },
        "currency": {
          "type": "string",
          "description": "Currency code
following the ISO 4217 standard. Ex: US
currency code is 'USD'.",
          "pattern": "^[A-Z]{3}$"
        },
        "brand": {
          "type": "string",
          "description": "NIKE brand",
          "enum": [
            "NIKE"
          ]
        },
        "channel": {
          "type": "string",
          "description": "Sales channel of
the shopping cart.",
          "enum": [
            "NIKECOM"
```

```
            ]
          },
          "items": {
            "type": "array",
            "description": "List of items.",
            "maxItems": 100,
            "items": {
              "type": "object",
              "properties": {
                "id": {
                  "type": "string",
                  "description": "Shopping cart
item unique identifier."
                },
                "skuId": {
                  "type": "string",
                  "description": "Product Stock
Keeping Unit (SKU) unique identifier."
                },
                "priceInfo": {
                  "type": "object",
                  "description": "Price details
for the cart or item (and valueAddedServices)
in whole. All price currencies are defined at
cart root.",
                  "readonly": "create-update",
                  "properties": {
                    "price": {
                      "type": "number",
                      "description": "Price of
```

```
the cart item.",
                      "readonly": "create-
update"
                    },
                    "subtotal": {
                      "type": "number",
                      "description": "Item
subtotal, the sku price multiplied by
quantity.",
                      "readonly": "create-
update"
                    },
                    "discount": {
                      "type": "number",
                      "description": "Item
discount.",
                      "readonly": "create-
update"
                    },
                    "valueAddedServices": {
                      "type": "number",
                      "description": "The total
cost of the value added services for the
item.",
                      "readonly": "create-
update"
                    },
                    "total": {
                      "type": "number",
                      "description": "Total
```

```
price of the item and value added services,
less any discounts.",
                      "readonly": "create-
update"
                    },
                    "priceSnapshotId": {
                      "type": "string",
                      "description": "The
price's snapshot in time unique identifier.",
                      "readonly": "create-
update"
                    }
                  },
                  "additionalProperties": true,
                  "required": [
                    "price",
                    "subtotal",
                    "discount",
                    "valueAddedServices",
                    "total",
                    "priceSnapshotId"
                  ]
                },
                "quantity": {
                  "type": "integer",
                  "minimum": 1,
                  "description": "Line item
quantity.",
                  "readonly": "create-update"
                },
```

```
              "valueAddedServices": {
                "type": "array",
                "description": "List of value
added services.",
                "items": {
                  "type": "object",
                  "properties": {
                    "id": {
                      "type": "string",
                      "pattern": "[0-9a-f]{8}-
[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-
f]{8}",
                      "description": "Value
added service unique identifier."
                    },
                    "instruction": {
                      "type": "object",
                      "description":
"Instructions for the value added service,
related to the various service domains.
Example would be a design id for Nike iD
customization.",
                      "properties": {
                        "id": {
                          "type": "string",
                          "description":
"Instruction unique identifier for the value
added service, related to the various service
domains. Example would be a design id for Nike
iD customization."
```

```
                        },
                        "type": {
                          "type": "string",
                          "description":
"Instruction Type, e.g. customization/nike_id
(only one currently available), customization/
my_print, customization/gift_card, buy/
gift_wrap, buy/gift_message",
                          "enum": [
                            "customization/
nike_id"
                          ]
                        }
                      },
                      "additionalProperties":
true,
                      "required": [
                        "id",
                        "type"
                      ]
                    },
                    "priceInfo": {
                      "type": "object",
                      "description": "Price
details for the value added service.",
                      "readonly": "create-
update",
                      "properties": {
                        "price": {
                          "type": "number",
```

```
                            "description":
"Price of the value added service.",
                            "readonly": "create-
update"
                          },
                          "discount": {
                            "type": "number",
                            "description":
"Value added service discount.",
                            "readonly": "create-
update"
                          },
                          "total": {
                            "type": "number",
                            "description":
"Total price of the value added service, less
any discounts.",
                            "readonly": "create-
update"
                          },
                          "priceSnapshotId": {
                            "type": "string",
                            "description": "The
price's snapshot in time unique identifier.",
                            "readonly": "create-
update"
                          }
                        },
                        "additionalProperties":
true,
```

```
                    "required": [
                      "price",
                      "discount",
                      "total",
                      "priceSnapshotId"
                    ]
                  }
                },
                "required": [
                  "id",
                  "instruction"
                ],
                "additionalProperties": true
              }
            }
          },
          "required": [
            "id",
            "skuId",
            "quantity"
          ],
          "additionalProperties": true
        }
      },
      "totals": {
        "type": "object",
        "description": "Price details for
the cart or item (and valueAddedServices) in
whole. All price currencies are defined at
cart root.",
```

```
            "readonly": "create-update",
            "properties": {
              "subtotal": {
                "type": "number",
                "description": "Subtotal of the
item costs for all items.",
                "readonly": "create-update"
              },
              "valueAddedServicesTotal": {
                "type": "number",
                "description": "Total of value
added services on the items.",
                "readonly": "create-update"
              },
              "discountTotal": {
                "type": "number",
                "description": "Total of all
discounts applied to the cart.",
                "readonly": "create-update"
              },
              "total": {
                "type": "number",
                "description": "Total price of
the entire item or cart, item costs less any
discounts.",
                "readonly": "create-update"
              },
              "quantity": {
                "type": "integer",
                "description": "Cart quantity.",
```

```
              "readonly": "create-update"
            }
          },
          "additionalProperties": true,
          "required": [
            "subtotal",
            "valueAddedServicesTotal",
            "discountTotal",
            "total",
            "quantity"
          ]
        },
        "errors": {
          "type": "array",
          "description": "List of errors in
the cart.",
          "readonly": "create-update",
          "items": {
            "type": "object",
            "properties": {
              "field": {
                "type": "string",
                "description": "The field
that has an error.",
                "readonly": "create-update"
              },
              "code": {
                "description": "Code for the
error, text based, 'screaming snake-case'. Ex:
'FIELD_INVALID'.",
```

```
                "readonly": "create-update",
                "type": "string",
                "enum": [
                  "REQUEST_INVALID",
                  "MISSING_REQUIRED",
                  "FIELD_INVALID",
                  "SKU_INVALID",
                  "PRODUCT_NOT_BUYABLE",
                  "ITEM_QUANTITY_LIMIT",
                  "QUANTITY_INVALID",
                  "SYSTEM_ERROR"
                ]
              },
              "message": {
                "type": "string",
                "description": "Plain text
description of the error.",
                "readonly": "create-update"
              }
            },
            "additionalProperties": false,
            "required": [
              "code",
              "message"
            ]
          }
        },
        "warnings": {
          "type": "array",
          "description": "List of warnings in
```

```
the cart.",
            "readonly": "create-update",
            "items": {
              "type": "object",
              "properties": {
                "field": {
                  "type": "string",
                  "description": "The field
that has a warning.",
                  "readonly": "create-update"
                },
                "code": {
                  "description": "Code for the
warning, text based, 'screaming snake-case'.
Ex: 'PRICE_CHANGED'.",
                  "readonly": "create-update",
                  "type": "string",
                  "enum": [
                    "PRICE_CHANGED"
                  ]
                },
                "message": {
                  "type": "string",
                  "description": "Plain text
description of the warning.",
                  "readonly": "create-update"
                }
              },
              "additionalProperties": false,
              "required": [
```

```
                "code",
                "message"
            ]
        }
    },
    "required": [
        "id",
        "country",
        "currency",
        "brand",
        "items"
    ],
    "additionalProperties": true
}
```

Idempotence Guarantee

In complex distributed systems, guaranteeing that an API request will be received only once by an application can be very difficult to achieve and validate when that application is hosted across geographic regions.

Our idempotence guarantee states that **subsequent duplicate requests to mutate the data will not change the state of the system**.

This can simplify the design when creating an eventually-consistent system, particularly when defining recovery scenarios that may need to replay API requests.

Security and Privacy

The security and privacy of your consumer's data is our #1 concern. Whether in-flight or at rest, your consumer's personally-identifiable information are protected according to the latest standards.

Well-Defined, Well-Documented

In addition to the guides on the Developer Portal, all NDe APIs have the following documents available at the root directory of the Bitbucket repository:

- Well-defined contract (API.md file) for each major version in API Blueprint format
- Request and response definitions in JSON Schema format within each API.md
- SLA.json file for response times
- Consistency guarantee for mutating services, e.g. ACID or Eventual (within API.md file)
- A README.md file including:
    - High-level explanation of the purpose of the API
    - Email address for support, maintenance, and feature requests
    - Current version of the API (major.minor)

    - Change log with changes between minor versions
    - Authentication and caching information for each endpoint
    - Description and example for each available ‘filter’ and query parameter
- A CHANGES.md file listing the change history

Prerequisites

Find out what you need in order to start using NDe APIs.

Registration

The NDe API registration process helps identify the software (including software version and who maintains it) that is calling an API.
This is useful for understanding the impact of changes to, or deprecation and removal of an API.
As of the time of this document, only an optional registration process exists, which is described below.

Caller Identification Process (Optional)

Caveats

For this process to work between your app and a particular NDe API, make sure that the answer is yes to all of the following questions:

1. Will you always call the API through api.nike.com?
2. Will you want your app to be identified to the API with a unique caller ID that might be visible on the public internet?
3. Since the caller ID might be visible to third-parties, will you allow your caller ID to be used by a different caller?

How does it work?

1. Register a caller ID with the Product Owner of the API, with the format of `<domain name>:<appid>:<majorVersionNumber>.<minorVersionNumber>`

2. In requests to that API, send your caller ID in request header **nike-api-caller-id**

Example caller ID: `com.nike:brand.ios.ntc:2.1`

Authorization

Many endpoints require that the customer has logged into their Nike account. This requires that your app prove that it is authorized to perform the requested operation on behalf of the customer by sending certain headers.

Depending on how you are calling the endpoint, the authorization-related headers you need to send will vary as follows:

Calls to the Nike API gateway (api.nike.com):

Send the access token in the Authorization request header that you obtained from Nike Unite/ Identity, prefixed by `Bearer` (note the single space after Bearer). Do not send the **upmid** header.

Required headers for calls to api.nike.com	Description
Authorization	Access token

Calls direct to endpoints:

Send the **upmid** header, and for those endpoints that require it, the **appid** header as well. Do not send the **Authorization** header.

Required headers for direct calls	Description
upmid	Nike user profile identifier
appid	Application identifier

TIP: To learn how to obtain an access token see the Generating an Access Token guide.

JWT (JSON Web Token)

Some endpoints such as *Submit Order Payments for Approval* require a JWT that is signed for a service authorized to call the endpoint.

In this case, pass the JWT in the **X-Nike-Authorization** request header. Also, send the name of the application (e.g. “checkouts”) that is authorized to call this endpoint in the **X-Nike-AppId** request header. This is the service name used to sign the JWT.

The JWT tokens are configured to be reusable within a certain time period, after which any calls using that JWT will be rejected. Work with the Product Owner of the API to understand the schedule for when the JWT token need to be updated.

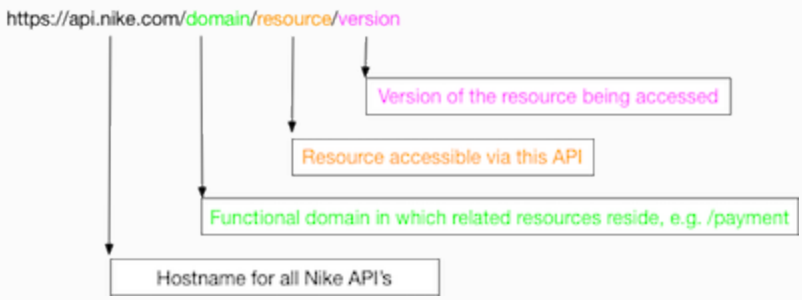
See the Nike JWT Reference Guide for more information.

Required headers for JWT	Description
X-Nike-Authorization	JWT token
X-Nike-AppId	Application identifier

TIP: You will need both a Production and Test JWT when calling JWT-required endpoints in those respective environments.

URL Patterns

The standard URL pattern used for NDe APIs (v2 or later) is as follows:



For example, all of Checkout APIs reside under the `/buy` domain, thus the URL's will always begin with https://api.nike.com/buy/.
The resource section of the URL varies depending on the endpoint, e.g. https://api.nike.com/buy/carts/ or https://api.nike.com/buy/checkout_previews/.

> **TIP:** Always check the API Developer Guide to confirm the correct URL format for a particular API.

Path Parameters

Path parameters are variable parts of a URL path. A URL can have one or more path parameters, each denoted with curly braces `{ }`.
For example, in the path /buy/carts/v1/{id} the id is the path parameter used to create or access a shopping cart resource.
NDe API path parameters are always required.

Query Parameters

Query parameters can be added to the end of the URL and they allow you to be more specific about what you are requesting.
Query parameters in NDe APIs, with one exception (see Filter below), are in the format of `?<name>=<value>`. Multiple query parameters can be chained together with ampersands like

`?<name>=<value>&<name>=<value>&<name>=<value>`.

For example, the query parameter named **marketplace** can be added to this Product Feeds request with the value 'US' as follows:

https://api.nike.com/product_feed/threads/v2/bcbeae50-28a5-404d-9941-fbbdff0c7860?marketplace=US.

The available query parameters vary per NDe API. They are sometimes optional, sometimes required, depending on the API. Check the Developer Guide for the API in question to confirm the query parameter requirements.

Common Query Parameters

There are few query parameters that are intended to be common across multiple NDe APIs. The first parameter, Fields, can be applied to any NDe API which returns a response body. The remaining parameters, Anchor, Count, Filter, and Sort, only apply to APIs which return a collection (i.e. a set of multiple results, not a single result).

> **TIP:** Not all NDe APIs support all the common query parameters. Check the Developer Guide for the API in question to confirm the query parameters supported.

Fields

Use the fields query parameter to select which fields will be included in the response.

Required format: `?fields=field1,field2,field3`

Nested fields are specified by parenthesis like `?fields=field1(field2)` or with dot notation like `?fields=field1.field2`, depending on the API.

An example from the *Create Or Update A User's Cart* endpoint of the Carts API:

https://api.nike.com/buy/carts/v1/61bc185b-16e5-43b5-bcaf-dd6168c543f8?fields=totals(total),totals(quantity) would return only the **total** and **quantity** fields nested under **totals**.

TIP: See API Standards for more info on using query parameters with NDe APIs.

Anchor

Use the anchor query parameter to request the previous or next page of a paginated collection. The anchor is provided to the client in a prior API call, to be used as a query parameter in a later call if desired.

Required format: `?anchor=<number>`

Count

Use the count query parameter to set the maximum number of elements to return in the response.

Required format: `?count=<number>`

Filter

Use the filter query parameter to restrict the API response based on one or more criteria. The API.md specifies which filter values (if any) are supported.

Required format: `?filter=filterName(filterValue)`

An example from the *Retrieve User Carts by Filter* endpoint of the Carts API:

https://api.nike.com/buy/carts/v1/?filter=country(US) would return only the carts created for a particular user with a country code of 'US'.

Chain more than one filter query parameter together using `&` in between each one, for example:

https://api.nike.com/buy/carts/v1/?filter=country(US)&filter=brand(NIKE)&filter=channel(NIKECOM) would return only the carts created for a particular user with a country code of 'US', a brand name of 'NIKE', and a channel name of 'NIKECOM'.

Sort

Use the sort query parameter to determine the ordering of the results in the response based on

one or more criteria.

Required format: `?sort=fieldNameAsc` (for Ascending order) OR `?sort=fieldNameDesc` (for Descending order)

For sorts of nested fields use dot notation like `?sort=fieldName.nestedFieldAsc`.

Example of sorting by a nested field:

https://api.nike.com/product_feed/feeds/v2?sort=publishedContent.publishStartDateAsc

To sort by multiple keys, use commas to separate them, for example `?sort=fieldOne,fieldTwoAsc`.

Example of sorting with multiple keys:

https://api.nike.com/product_feed/feeds/v2?filter=channelId(008be467-6c78-4079-94f0-70e2d6cc4003)&sort=publishedContent.publishStartDateAsc,id.keywordAsc

TIP: Always check the Developer API Guide for the specific implementation of path and query parameters for a particular API.

Request Components

Read about the common components of HTTP requests sent to NDe APIs. Also see the related Response Components section.

URI (Universal Resource Identifier)

By sending a request to a NDe API, you access a resource using a particular URI (Universal Resource Identifier) comprised of a string of characters.

For more info, see the URL Patterns section.

HTTP Methods

Access API resources via standard HTTP methods, noting that not all APIs support all methods.

Method	Description
GET	Access an existing single resource or collection of resources (Idempotent)
POST	Create a new resource
PUT	Update an existing resource by passing entire resource in payload (Idempotent)
DELETE	Delete an existing resource (Idempotent)
PATCH	Update an existing resource by passing partial updates by passing only affected fields are passed in the payload

Examples:

```
GET /persons -- List all persons
PUT /persons -- Bulk update persons
POST /persons -- Add a new person
DELETE /persons -- Delete all persons
```

Request Headers

The required request headers vary per API and are described in the detailed per-API guides.

This section describes the commonly-used headers by category.

General Headers

Header	Description
accept	Content type you will accept in response, application/json is only value allowed
content-type	Content type of the request, application/json is only value allowed
true-client-ip	IP address of the client, required if **X-Forwarded-For** is null
x-forwarded-for	Used to derive the IP address of the client, often required if **True-Client-IP** is null
user-agent	Browser and operating system of the client calling this endpoint
upmid	Nike profile ID of customer, required if **x-nike-visitorid** is null
x-nike-visitorid	Visitor id of a non-member, required if **upmid** is null
usertype	‘nike:swoosh’ for Nike Employees, ‘nike:plus’ for Nike+ Customers, otherwise do not send
origin-order-id	During v1 to v2 cutover, represents the legacy order id associated to the checkout and is used by downstream systems

Authorization Headers

Header	Description
Authorization	Access token
appid	Identifier of calling application
X-Nike-Authorization	JWT token
X-Nike-AppId	Application identifier (when JWT required)

See the Prerequisites section for more info on Authorization headers.

Request Body

Format

The default request body format is JSON (application/json) with charset UTF-8.

Detailed request formats, including required and optional fields, are described in JSON Schema in each API contract (API.md file) and in the detailed per-API guides.

> **TIP:** GET requests do not require a request body.

Below are a few of the general rules, but **always follow the JSON Schema mentioned in the API.md**.

Field Names

Field names are in camelCase, for example:

- firstName
- postalCode
- startTime

Enumerations

Use SCREAMING_SNAKE values for enumerations, for example:

Element	Enumeration values
code	“REQUEST_INVALID”, “MISSING_REQUIRED”, “FIELD_INVALID”, “SKU_INVALID”,
status	“PENDING”, “IN_PROGRESS”, “COMPLETED”

Query Parameters

Query parameters can be used in the URI as per the below table.

Not all parameters are supported by all APIs. See the API.md or the detailed per-API guide to know which are supported by that API.

Query Param	Description
anchor	In a collection, return elements after the anchor at the end of the previous page
count	Maximum number of elements to return
fields	Select which fields to be included in response
filter	Filter results based on a provided element value
sort	Sort returned results

See the URL Patterns section for more info about how to use these query parameters.

Making Your First API Request

See the detailed per-API guides for examples of how to make your first API request!

Response Components

Learn about the common components of HTTP responses returned by NDe APIs. Also see the related Request Components section.

HTTP Status Codes

The HTTP protocol defines status codes to clearly describe the result of an API call. Here is a sampling of standard response codes you may see in responses:

Code	Description
200 (OK)	Standard success response, including PUT/PATCH
201 (Created)	Standard success response for POST (create resource)
202 (Accepted)	Standard response for long-running async processing
204 (No Content)	Successful DELETE response
301 (Moved Permanently)	URL for a resource was moved (different host name or even non-supported versions)
304 (Not Modified)	Used with ETags
400 (Bad Request)	Request badly-formatted or not following the correct schema
401 (Unauthorized)	Not a valid access token
403 (Forbidden)	Valid access token, but access is

Code	Description
	forbidden to requested resource
404 (Not Found)	Resource not found
406 (Not Acceptable)	Resource format/type not supported (e.g. XML)
409 (Conflict)	Resource could not be updated due to other updates conflicting
412 (Preconditioned Failed)	Request headers did not meet the requirements
413 (Request Entity too Large)	Request body too large, like over 1 MB
429 (Too Many Requests)	Number of allowed calls was exceeded per defined time interval (day or hour or minute)
500 (Internal Server Error)	Server error (details provided in the body)
503 (Service Unavailable)	Service is down

Response Headers

Every API response includes headers, but the headers sent will vary. Some of the typical headers for NDe APIs are described below by category:

General Headers

Header	Description
cache-control	Tells caching mechanisms from server to client whether they may cache this object. Measured in

Header	Description
	seconds
content-encoding	Type of encoding used on the data, e.g. gzip
content-type	Indicates the media type of the entity-body returned
content-length	Length of the response body in bytes
date	Date/time that the message was sent
expires	Date/time after which the response is considered stale
server	Name for the server
set-cookie	HTTP cookie
status	Status of HTTP response
vary	Tells downstream proxies how to match future request headers to decide whether the cached response can be used rather than requesting a fresh one from the origin server.

CORS (Cross-Origin Resource Sharing) Headers

If your API request requires a CORS pre-flight message to be sent before the actual request, you may receive the following response headers:

Header	Description
access-control-allow-credentials	Indicates whether or not the actual

Header	Description
	request can be made using credentials
access-control-allow-headers	Indicate which HTTP headers can be used when making the actual request
access-control-allow-methods	Specifies the method or methods allowed when accessing the resource
access-control-allow-origin	Specifies a URI that may access the resource
access-control-expose-headers	Lets a server whitelist headers that browsers are allowed to access

See the CORS section for more info.

Custom Headers

NDe APIs may use the following custom headers:

Header	Description
x-b3-traceid	TraceId that will be carried through all the distributed systems for a request, e.g. c2f80be958b69372
x-nike-application	Nike API application name, e.g. productfeed
x-nike-environment	Nike environment, e.g. prod
x-nike-version	Nike API version number, e.g. 1.0.0.179

TIP: For more info on how to use the **x-b3-traceid** header for troubleshooting, see the Troubleshooting section.

Example of actual response headers from the Product Feeds API:

```
access-control-allow-credentials:true
access-control-allow-
headers:Accept,access,Authorization,Content-
Type,cloud_stack,Origin-Order-ID,x-nike-
visitorid,x-nike-visitid,nike-api-caller-id,X-
B3-TraceId,X-B3-SpanId,X-B3-ParentSpanId,X-
B3-SpanName,X-B3-Sampled
access-control-allow-
methods:GET,POST,PUT,OPTIONS,DELETE,PATCH
access-control-allow-
origin:https://www.nike.com
access-control-expose-headers:Date,WWW-
Authenticate
cache-control:private, no-transform, max-age=28
content-encoding:gzip
content-length:26936
content-type:application/json; charset=UTF-8
date:Fri, 22 Sep 2017 17:36:43 GMT
expires:Fri, 22 Sep 2017 17:37:11 GMT
server:Jetty(9.2.15.v20160210)
set-
cookie:ak_bmsc=48F39E22CBDF84176C09B5C095EE587E
```

```
B832586DFC6C00002B4AC559E110096D~pltyB9LKTVZq1Q
cwSkpJMTSSzPqQP+XSvOCNcDRh7Uaizce2BiYaZ3QV4
CuxZdUV0Tnsu5XPCRFBbHNV7k3gfMGL8v70jStJpQBBAGfJ
1VLVkRVV8ItIGqwtMtCxHWSwZwIEqELJ+4YMqFXNgYkI5RN
wNUpmDVYIe0S1EhX7ZIYEmKgig2d8eEHtP6Qt8zwlNLuExK
GF5pMbSwe3PqaN6SK3bCPSdIRBbJ6X0Y90c=;
expires=Fri, 22-Sep-2017 19:30:43 GMT; max-
age=7200; path=/; domain=.nike.com; HttpOnly
status:200
vary:Accept-Encoding
x-b3-traceid:c2f80be958b69372
x-nike-application:productfeed
x-nike-environment:prod
x-nike-version:1.0.0.179
```

Response Body

Your responses will usually include a body but there are certain HTTP methods that don't require response bodies to be sent.

The default response body format is JSON (application/json) with charset UTF-8.

Detailed response formats are described in JSON Schema in each API contract (API.md file) and in the detailed per-API guides.

Error and Warning Messages

Although NDe uses a standard formatting for error and warning messages included in a response body, the content of each is specific to an API endpoint.

See the per-API guides in each endpoint section for detailed error/warning information.

For additional general error handling info, see the Error Handling section.

Using the API Reference

This section describes how to use the NDe API Reference documents.

Overview

Each NDe API Reference document serves as the contract for using the API, and has the following features at minimum:

1. Describes all available endpoints
2. For each endpoint, lists which URI query and path parameters are required or optional, including data types and sample data
3. For each endpoint, lists which types of requests and responses are allowed or expected, including headers and payload details with sample data and JSON schema for each

> **TIP**: In some cases, depending on the API, the doc will also provide

details for authorization/ authentication, error code information, or an overview of the API.

Accessing the API Reference on the Developer Portal

The steps for accessing the API Reference on the Developer Portal are as follows:

1. Navigate to the Developer Portal
2. Use the Search bar or the Service Catalog link to locate the API
3. Click the **API** tab to display the API Reference document
4. Scroll to the endpoint you are interested in
5. The **URI PARAMETERS** section is expanded by default
6. To expand additional sections, click **SHOW** on the line of the section you wish to display
7. Each request/response section may have a **Headers**, **Body**, **Schema** subsection
8. Scroll to the subsection of interest to display it
9. Within a **Schema** subsection, any required fields are either listed in a “Required” section or indicated at the field level

TIPS:

✔ For more on JSON Schema, see the JSON Schema Helps Define API Contracts section of this doc.

✔ For more on how to use the Developer Portal, see the Developer Portal User Guide

✔ Another option for getting details of an API is the Developer’s Guides. Note not all APIs have

Developer's Guide at this time.

Versioning

As NDe APIs are enhanced over time to add new features and fix bugs, the version numbers are incremented according to Semantic Versioning guidelines.

Some high-level considerations:

- For minor version increments or patches, e.g. the addition of a new, optional field, the changes are non-breaking and the endpoint URL does not change. If you are using the Tolerant Reader Pattern, you can continue to use the API without having to make changes to your app.
- When moving to a new, major version of an API (e.g. a change from synchronous to asynchronous operation), expect to make some changes to your app to ensure compatibility with the new version.

TIP: For more details about the versioning of NDe APIs, see the API Versioning Strategy document.

Caching

NDe APIs take advantage of three layers of caching in order to keep service performance optimal even in periods of high request volume. The caching layers are:

1. Akamai Content Delivery Network (CDN)
2. Service
3. Device/Browser

Akamai Caching

NDe uses the Akamai Content Delivery Framework as the Edge caching solution for public service requests. It is utilized when the client makes a request for a NDe public resource configured to go through Akamai's Edge server. Akamai caching and routing is managed though a set of configurations at Akamai. Akamai caching is bypassed in application to application calls because the requests do not go through Akamai.

It is referred to as an Edge server because it is on the Edge of two networks, in this case the public internet and Nike's Edge router. Akamai operates on a set of configured rules that determine what resources are cachable, how long to cache the resource and how to determine if the resource is stale and if the origin of the resource has an updated version. Akamai retrieves a cached copy of the data that is as close to the caller as possible to ensure the quickest response time.

Listed below are the Production domains that are routed to Akamai's Edge caching server:

Domain	Description
www.nike.com	For unauthenticated, public experiences, services and assets
api.nike.com	For authenticated public experiences

Service Caching

NDe Architecture encourages caching at the individual service level and discourages implementing custom distributed caching solutions, due to high development costs. Not all NDe Cloud services support caching and cache times vary across services. Check the Caching section of the NDe API Documentation you are interested in for cache information by service.

Device/Browser Caching

Caching can also be done on the Browser/Phone device itself. This type of caching is controlled

by exchanging a series of request and response headers as is demonstrated in the flow below.

1. The client sends a service request for a URL resource.
2. The service returns the resource response with an **ETag** header representing the version of the data, an **Expires** header representing how long until the URL expires, and a **Cache-Control** or **Pragma** header indicating that the response can be cached.
3. The client caches the response and **ETag** value.
4. If the client needs the same resource again, the client uses the **Expires** and **Pragma**/**Cache-Control** headers to check if the local URL cache has expired.
5. If the client cache has not expired, the client retrieves the local version from cache.
6. If the client cache has expired, the client sends a service request for the previously requested URL resource, sending the **ETag** header and **If-None-Match** header set to the cached ETag value.
7. Because the **If-None-Match** header is present, the Service checks its **ETag** value with the **ETag** value passed in the request header to check if it has a newer version of the resource.
8. If the **ETag** values match, the service returns a shortened response and with an HTTP 304 Not Modified status, greatly reducing network bandwidth.
9. If the **ETag** value doesn't match, the service returns the full response and an updated value in the **ETag** header and the process restarts.

Cache-Related Headers

Request

Header	Description	Example
Cache-Control	Directive indicating what and how resource can be cached	
If-None-Match	Used on request with ETag (sent prior by server) to only get	If-None-Match: "686897696a7c876b7e"

Header	Description	Example
	resource if newer than tag or 304 otherwise	
If-Match	Used with the value from the received ETag header for PUT/ PATCH to achieve optimistic locking (conditional update)	
ETag	String that identifies a specific “version” of the requested, mutable resource.	ETag: “686897696a7c876b7e”
Pragma	Used to disable caching for a resource or at least let the client know this resource must not be cached	Pragma: “no-cache”
Expires	Date indicating when this resource is stale. Do not use the Pragma with this header	Expires: Fri, 01 Dec 2017 16:00:00 GMT

Response

Header	Description	Example
ETag	String that identifies a specific “version” of the mutable resource.	ETag: “686897696a7c876b7e”
Expires	Sate indicating when	Expires: Fri, 01 Dec

Header	Description	Example
	this resource is stale. used with **Cache-Control**	2017 16:00:00 GMT
Cache-Control	Used for enabling caching of resources and identifying what can be cached. used with Expires.	Cache-Control: private,max-age=0
Pragma	Used to disable caching for a resource or let the client know this resource must not be cached. use **Cache-Control** instead.	Pragma: “no-cache”

TIP: See these related links for more info:

⚯ Nike Caching Strategy

⚯ Akamai Technical Reference

CORS

Client-side HTTP requests are subject to the same-origin policy, meaning the requested resource must be for the same domain, port, and protocol as the originator of the request. This restriction prevents unsafe requests that could compromise data integrity. For instance, a request from a script on api.nike.com for an image on images.nike.com violates the same origin

policy and results in a security error.

One way to relax this restriction is to make CORS requests. CORS (Cross-origin resource sharing) gets around this restriction through an exchange of headers between the browser making the request and the server of the requested resource. The browser sends an Origin header containing the origin making the request (e.g. http://api.nike.com) and the server sends a Access-Control-Allow-Origin header in the response that lists all origins allowed to access it. If the origin is in the list, the browser lets the request through. For a detailed explanation of CORS and implementation examples see HTTP Access Control (CORS).

Implementation Recommendations

Developers implementing Cloud Services should keep the following best practices in mind:

- Be consistent and as strict as possible in allowed CORS header values.
- Avoid using the wildcard character * as Access-Control-Allow-Origin response header value. Use a list of specific values instead.
- Control restriction of Access-Control-Allow-Headers, Access-Control-Expose-Headers, Access-Control-Allow-Methods, and Access-Control-Allow-Origin headers for both requests and responses at the service level, perhaps through the use of a Servlet Filter.
- Avoid setting Allow-Control-Allow-Credentials to true unless the service requires cookies. This value is false by default.

Asynchronous Operation

A **synchronous** endpoint returns the result immediately because both the request and work performed execute in the same thread. Synchronous endpoints are quick-running.

An **asynchronous** endpoint takes a work request and promises to return the results in an estimated time in the future. It immediately returns a status, an eta, and a result link at which the caller should poll for job results. It is best practice to wait the duration of the eta before asking for job results, also known as Planned Polling. If the job still is not finished, the caller should wait for the length returned in the job result eta before polling the job result again.

It is suggested that services running longer than 250ms be an asynchronous service; it is mandated that services running longer than 500ms be an asynchronous service.

Asynchronous jobs are either in PENDING, IN_PROGRESS, or COMPLETED status. PENDING indicates that the job has been accepted into the work queue. IN_PROGRESS indicates the job has been picked up from the work queue and is being actively worked on. COMPLETED indicates the job is complete and results are available.

There are two or three endpoints involved in an asynchronous service: job request, job status and job result. Job status and job result endpoints are sometimes combined.

Job Request

This endpoint returns a unique job id in UUID format, a status, eta of when the job will be finished and a **self** link to the job result to poll for job status. Notice that the **self** link contains the same UUID as the job id. In this case, the caller should wait 3000ms before polling the job result link.


```
{
        "id":
"5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "PENDING",
        "eta": 3000,
        "resourceType": "job",
        "links": {
                "self": {
                        "ref": "/buy/checkouts/
v2/jobs/5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        }
}
```


Job Status

This endpoint is called by passing the **id** returned form the Job Request response above.

Because the job status is not `COMPLETED`, the calling service should wait 500ms before polling the job status again.

```
{
        "id":
"5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "IN_PROGRESS",
        "eta": 500,
        "links": {
                "self": {
                        "ref": "/buy/checkouts/
v2/jobs/5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        },
        "resourceType": "job"
}
```

Job Result

The job is now `COMPLETED` and resulted in an error. The errors array lists details about each error including the error message and error code.

```
{
        "id":
"5f650e83-ed55-44d5-9820-650e8c390c7e",
        "status": "COMPLETED",
        "error": {
                "message": "Authorization
failed for order C00013575031",
```

```
                "httpStatus": 409,
                "code": "PAYMENT_INVALID",
                "errors": [{
                        "message": "Payment
failed",
                        "code":
"PAYMENT_INVALID"
                }]
        },
        "links": {
                "self": {
                        "ref": "/buy/checkouts/
v2/jobs/5f650e83-ed55-44d5-9820-650e8c390c7e"
                }
        },
        "resourceType": "job"
}
```


Error Handling

Use this guide to understand what to expect from NDe API error responses and learn some tips on how to handle them.

General Error Response Components

Error responses from NDe APIs contain the following:

- HTTP Status Code

  NDe uses standard HTTP status codes for errors. See the Common Errors section for more details.
- Response Body Components

Element Name	Description
message	Plain text description of the error at

Element Name	Description
	top level of response
errors	Array at top level of response, containing the following:
code	Code for the error, text-based used ‘screaming snake case’, e.g. ‘FIELD_INVALID’
message	Plain text description of the error
field	Field in the request which had the error, in JSON Pointer format

Note: not all HTTP responses include a body, e.g. 204 or 304.

- Response Header Components

  NDe APIs return a Trace ID in the **X-B3-TraceId** response header. This can be used to query logs to troubleshoot the error.

TIP: For more about how to use Trace IDs, see the Troubleshooting section.

Common Errors

Examples error responses are provided below, along with the status code and scenario in which they occurred. A few considerations:

- Some of the examples are API-specific and therefore cannot be assumed to be universal to all NDe APIs.
- Not all HTTP responses include a body, and those are indicated below.

1. **400 - Bad Request:** Invalid JSON

```
{
    "message": "Validation Failed",
    "errors": [
        {
            "message": "Invalid request body",
            "code": "REQUEST_INVALID"
        }
    ]
}
```

Notes: Fix the syntax of the JSON request and retry.

1. **400 - Bad Request:** Required fields missing

```
{
    "message": "Validation Failed",
    "errors": [{
        "field": "/fieldName",
        "code": "MISSING_REQUIRED",
        "message": "Required field"
    }]
}
```

Notes: Add the mentioned required fields to the JSON request and retry.

1. **400 - Bad Request:** Field data invalid

```
{
    "message": "Validation Failed",
    "errors": [{
```

```
        "field": "/email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

Notes: Correct the mentioned field data and retry.

1. **401 - Unauthorized:** User not authenticated

```
{
    "error_id":
"1ea29a72-52c1-46f3-83b3-2c93684fa4c9",
    "errors": [
        {
            "code": 35,
            "message": "Unauthorized user
access"
        }
    ]
}
```

Notes: check that a valid access token was sent in the **Authorization** header. Correct and retry.

1. **403 - Forbidden:** User authenticated, but operation not allowed

```
{
    "httpStatus": 403,
    "code": "77773",
    "timestamp": 1508186768127,
```

```
    "service": "paymentapproval",
    "message": "Authorization failure",
    "errors": []
}
```

Notes: check JWT token (if applicable) and retry.

1. **404 - Not Found:** Resource does not exist

```
{
    "error_id": "3c63d8ff-
bb08-4331-b89c-0bc6b4695d9a",
    "errors": [
        {
            "code": 310,
            "message": "The resource you
requested does not exist."
        }
    ]
}
```

1. **405 - Method Not Allowed:**

(no response body sent for this error)

Notes: check API documentation for supported methods

1. **406 - Not Acceptable:**

(no response body sent for this error)

Notes: check **Accept** header is present and set to ‘application/json’ and retry.

1. **409 - Conflict:** Operation failed

```
{
    "message": "Request conflicts with
previous request for same checkout",
    "code": "REQUEST_INVALID"
}
```

Notes: retry with a never-used ID in the request (if applicable).

1. **500 - Internal Server Error:** Service had an exception

```
{
    "httpStatus": 500,
    "timestamp": 1500926568225,
    "service": "cartreviews",
    "message": "Server error"
}
```

1. **503 - Service Unavailable:** Service is down

(no response available)

Which JSON Field Had The Error?

NDe uses the JSON Pointer standard to indicate which field of the request JSON had the error. For example, in the error response from the Carts API you can see the field indicated as `/request/items/0/contactInfo/email`:

```
{
    "message": "Validation Failed",
    "errors": [{
```

```
        "field": "/request/items/0/contactInfo/
email",
        "code": "INVALID_EMAIL",
        "message": "Invalid email"
    }]
}
```

The field names are separated by `/` to indicated nesting in the structure of the request JSON.

TIP: Some NDe APIs use dot notation instead of JSON Pointer, due to being built before NDe switched to the JSON Pointer standard. Check the Developer Guide for the API to confirm the error format.

Retries

Depending on the returned HTTP status code, retrying an operation might make sense or not.

In general, a 400-class status means a client-side issue, while a 500-class status means a server-side issue.

Here are some recommendations for retries:

HTTP Status Code	Description	Retry?	Comments
400	Bad Request	Yes - after changes	Change JSON based on response, retry

HTTP Status Code	Description	Retry?	Comments
			operation
401	Unauthorized	Yes - after login	Once user logs in, retry operation
403	Forbidden	Yes - after changes	Once correct permissions of logged-in user are granted, retry operation
404	Not Found	No	
405	Method Not Allowed	No	Change app to call API as per spec
406	Not Acceptable	No	Change app to call API as per spec
409	Conflict	No	JSON input and query params are ok but a business rule denied the operation. No need to retry
500	Internal Server Error	No	Server issue. No need to retry until issue is resolved
503	Service Unavailable	Yes	API may come back up. Retry a few times using

HTTP Status Code	Description	Retry?	Comments
			circuit breaker pattern

NOTE: The JSON Schemas found in the API.md file should enumerate many of the possible error messages you could receive in responses. However, JSON Schemas do not include all possible error codes or messages. Specific errors can be added by each domain and are not part of these schemas.

Data Reference

User Types

NDe APIs support 3 distinct user types for commerce applications. In this guide, we will define them and discuss some ways that it can affect how you interact with the APIs.

Member

Nike members have previously registered a Nike+ account and have logged in with their credentials from inside your app. Members get benefits like free shipping, free 30-day trials, and the ability to save shipping and payment information for faster checkout.
For API calls involving members, an *access token* must be obtained from Nike Unite services and included in the **Authorization** request header after the user has logged in.

Once Nike has validated the access token, the APIs will automatically adjust behavior as necessary based on the knowledge that the user is a member and based on our business rules.

Guest

The guest user has not logged in with their Nike+ account credentials, effectively making them a new, anonymous user to Nike. When making a purchase, the guest user must input all of their information from scratch and does not receive the additional benefits that a member would.

For API calls involving guests, the **nike-visitor-id** and **appId** headers must be included with the request. The **nike-visitor-id** header value is a UUID that you get by calling Nike Unite's getVisitData function in their SDK. The **appId** header value is the identifier for your app.

Employee

The third type of user is an employee of Nike or one of it's subsidiaries (or an immediate family member of said employee) who has logged in with their Swoosh account credentials. The employee user receives special pricing on most products and may be offered different shipping options than a member or guest.

For API calls involving employees, the same **Authorization** request header is used like for members.

Testing

Testing Prerequisites

The same prerequisites for interacting with APIs in the production environment also apply to test environments, i.e. registration, authorization, and JWT. Note that tokens are often unique to an environment, so your production tokens will not work in test and vice versa.

See the Prerequisites section for more info.

Test Environments

The ability to test in an environment other than production varies per API. For many APIs,

functional test environments exist and should be used. For others that do not, testing in production may be an option.
Contact the Product Owner of the API (listed in each Developer API Guide, At-a-Glance section) for recommendations on the best environment to use for testing.

Testing Tips

When testing isolated API calls from your local machine, here are a few tips:

- Use the Nike Developer Portal 'TRY IT NOW' feature to initiate calls from within a browser.
- Use the Postman REST client or a similar tool to test sending HTTP requests.
- Locate a small set of production skuId's to use. Reuse them whenever possible.
- For Checkout API's, use the same set of line item 'id' values for all requests. They only need to be unique with a particular request.
- Use an online UUID generator to quickly generate UUID's.
- To save time, modify the sample requests in the API guide rather than building your own.

Troubleshooting

Stuck on something with a particular API? See the below troubleshooting tips.

Query Logs With a Trace ID

If you get unexpected or confusing responses from an API, use the Trace ID from the response header to query the Nike CDT Splunk logs to get more information.

1. Find the **X-B3-TraceId** response header, which contains the Trace ID for the request, e.g. b2490b12cd639e7c. Copy the value to your clipboard.
2. Navigate to Splunk Search Page.
3. Paste in the Trace ID and execute the query. Once results start coming in, look for error messages or other clues as to what happened with your request.

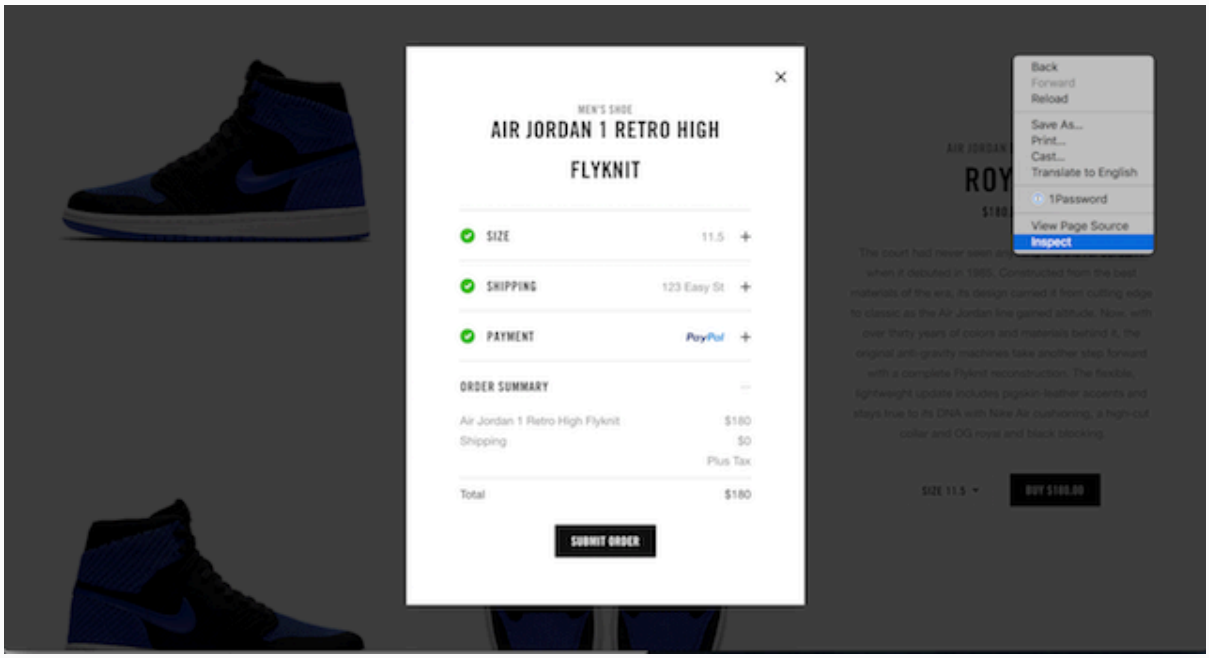
After looking at Splunk, if you still need assistance with troubleshooting a particular request, post a question on the relevant Slack channel (found in the Developer API Guide) and be sure to include your Trace ID.

Inspect Browser Activity in a Live Experience

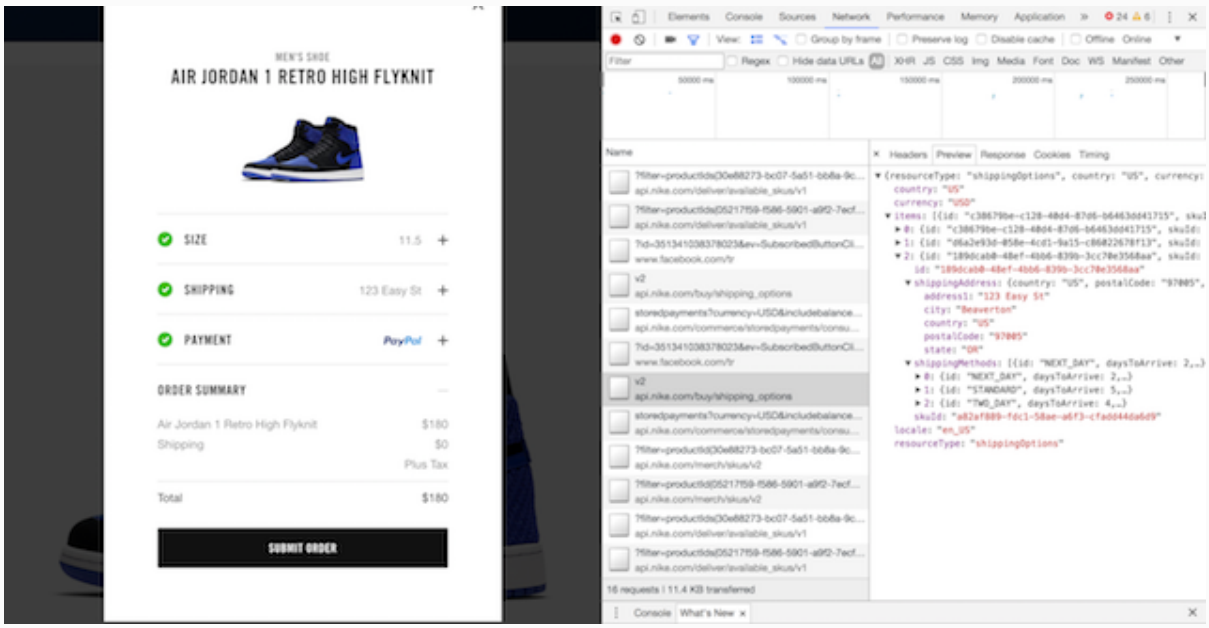
Use your browser's built-in tools for inspecting the web service calls which occur for a live Nike experience like Nike SNKRS Web. Sometimes seeing what other experiences are doing might address your question or concern.

For example, to follow the order of calls made when changing a shipping address during checkout:

1. Right-click anywhere in browser main window, select `Inspect`.



1. In Inspect window, select `Network` tab.
2. Perform the action in the main window. In this case, change the shipping address and click 'Save and Continue'.
3. In the `Network` tab, scan through the list for any items with "api.nike.com". In this case, click to select the call to "api.nike.com/buy/shipping_options".



1. Study the data in the Headers, Preview, and Response tabs. Is there some request header data present that you hadn't considered? Is the data in the request body or response body as expected?

Circuit Breaker Best Practices

Be a good client by following these circuit breaker best practices when calling Nike APIs.

Glossary

For a master glossary of terms for Nike APIs, see the Glossary.

Related Links

NDe Documentation Home

Getting Started

Business Guides

Developer's Guides